

Al-Aqsa University

Faculty of Computers and IT

Engineering and Networks Dept.



جامعة الأقصى
كلية الحاسبات وتكنولوجيا المعلومات
قسم الهندسة والشبكات

Operating Systems for Intelligent Systems (ENGS2306)

Lecture 9: The Final Project - Automating the IoT Monitoring Hub

Lecturer: Eng. Mohammed E. Alhams

Semester: Second Semester 2024/2025

Lecture 9: The Final Project: Automating the IoT Monitoring Hub

Today's Objectives:

1. Review the core skills you have mastered throughout the course.
 2. Introduce the Final Project scenario: building an automated "IoT Monitoring Hub".
 3. Learn the few new "just-in-time" tools needed to complete the project (apt, systemctl, ufw).
 4. Understand the project requirements, deliverables, and how you will be assessed.
-

The Road to the Final Project: What You Already Know

This project is designed to bring together everything you have learned. You already have 90% of the skills you need.

- **Phase 1 (Foundations):** You mastered the command line and file system (ls, cd, mkdir, cp, echo).
- **Phase 2 (Core Admin):** You learned to manage users, groups, and permissions (useradd, usermod, chown, chmod). You also learned to manage processes (ps, pgrep).
- **Phase 3 (Automation):** You learned to write powerful scripts with variables, if statements, functions, and error checking (set -e). You also learned the basics of networking (ssh).

Today, we add the final few tools to complete your toolkit.

The Project Scenario: The IoT Monitoring Hub

Imagine you work for a company that deploys hundreds of IoT devices. You need a central "hub" server for these devices to report their status. Setting up each hub manually is slow and leads to errors.

Your mission: Write a single Bash script that completely automates the setup of a new hub on a fresh Ubuntu server.

The script must handle everything: creating users, setting up directories, installing software, configuring security, and verifying that the deployment was successful.

New Tool 1: Installing Software (apt)

To create a hub, we first need to install web server software. In Ubuntu, we use the **Advanced Package Tool (apt)**.

- `sudo apt update`: Resynchronizes the package index files from their sources. Always run this first.
- `sudo apt install <package_name>`: Installs a new package.

For Automation: A normal install will ask "Do you want to continue? [Y/n]". To prevent this, we use the -y flag.

Command for your script:

```
# This command will install the Nginx web server without asking for confirmation.
sudo apt install nginx -y
```

New Tool 2: Managing Services (systemctl)

Once software is installed, it runs as a **service** (or daemon). systemctl is the command to manage these services.

- systemctl start <service>: Starts a service immediately.
- systemctl status <service>: Checks if a service is running.
- systemctl enable <service>: **Crucially**, this makes the service start automatically every time the system boots up.

Commands for your script:

```
# Make nginx start on boot
sudo systemctl enable nginx

# Start nginx right now
sudo systemctl start nginx
```

New Tool 3: Basic Firewall (ufw)

A server connected to the internet must be protected by a firewall. We will use **Uncomplicated Firewall (ufw)**.

The logic is simple: block everything first, then allow only what you need.

Commands for your script:

```
# Deny all incoming connections by default
sudo ufw default deny incoming

# Allow all outgoing connections
sudo ufw default allow outgoing

# Allow SSH (so you don't lock yourself out!)
sudo ufw allow ssh

# Allow HTTP traffic for our web server
sudo ufw allow http
```

```
# Enable the firewall (use -y for non-interactive confirmation)
echo "y" | sudo ufw enable
```

The Project Task Overview

Your final script (deploy_hub.sh) must perform these actions in order:

1. **Create** a dedicated user (iot_admin) and group (hub_admins).
 2. **Create** a web content directory (/srv/iot_hub) and set permissions so only the new user can manage it.
 3. **Install** the nginx web server.
 4. **Configure** the firewall (ufw) to only allow SSH and HTTP traffic.
 5. **Deploy** a custom index.html file to your web directory.
 6. **Enable and Start** the nginx service.
 7. **Verify** that the nginx process is running and print a final success or failure message.
-

Final Deliverables

This project is your final assessment. You will submit two files:

1. **deploy_hub.sh**: Your complete, well-commented Bash script. Your code should be clean, logical, and robust.
 2. **final_project_report.pdf**: A professional report containing:
 - The full text of your script.
 - A clear explanation of what each section of your script does.
 - Screenshots showing your script running successfully.
 - A screenshot proving the hub is working (e.g., using curl http://localhost).
-

Tips for Success

- **Develop Incrementally**: Don't try to write the whole script at once. Write one section (e.g., user creation), test it manually, then add it to your script.
- **Use Comments**: Explain *why* you are doing something, not just *what*. This is crucial for a good grade.
- **Use set -e**: Put set -e at the top of your script to make it automatically stop if a command fails. This will save you a lot of debugging time.

- **Start Early:** This is a comprehensive project. Do not leave it until the last day.

This is your chance to show everything you have learned. Good luck!

Course Wrap-Up

This project is the culmination of our 10-week journey. We started with the basics (ls, cd) and have now progressed to writing a complete automation script that manages users, permissions, software, and security.

The skills you have learned here are the foundation of modern system administration, DevOps, and managing intelligent systems.

Thank you for your hard work and dedication. Now, go build an amazing project!